Final project solution

1. Verification plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	when reset is low the output signals must be low except the empty flag must be raised	direct in the start of the test	-	immediate assertion in the top module
FIFO_2	when reset is low internal pointers and counter must be reset to zero	direct in the start of the test	-	immediate assertion in the design module
FIFO_3	check that internal pointers and conter not exceed the FIFO_DEPTH value	constrained randomization	-	immediate assertion in the design module
FIFO_4	check that data_out is reading properly when read enable raised while fifo is not full yet	constrained randomization	-	compare with a golden model
FIFO_5	check that data_out is not reading when the read enable raised but the fifo is empty	constrained randomization	-	compare with a golden model
FIFO_6	check that data_out is not reading when the read and write enable are raised simultaineuosly while fifo is empty and only the write operation is done	constrained randomization	-	compare with a golden model
FIFO_7	check that only read operation is done when the read and write enable are raised simultaineuosly while fifo is full	constrained randomization	-	compare with a golden model
FIFO_8	check that raising of the wr_ack signal when a valid write operation is done	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the design module
FIFO_9	check raising of the full signal when the fifo become full	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the design module
FIFO_10	check raising of the empty signal when the fifo become empty	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the design module
FIFO_11	check raising of the almost full signal when the fifo still have one empty location only	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the design module
FIFO_12	check raising of the almost empty signal when the fifo still have one data location only	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the design module
FIFO_13	check raising of the overflow signal when a write operation considered to be done while the fifo is full	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the design module
FIFO_14	check raising of the underflow signal when a read operation considered to be done while the fifo is empty	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the design module
FIFO_15	check for pointer wrapping	constrained randomization	-	concurrent assertion in the design module
randomization constraints: reset is 2% on, read enable is 30% on, write enable is 70% on				

2. Design file

```
import shared_pkg::*;
module FIFO (
    FIFO_if.DUT intf
);
  localparam max_fifo_addr = $clog2(FIFO_DEPTH);

  logic [FIFO_WIDTH-1:0] mem[FIFO_DEPTH-1:0];

  logic [max_fifo_addr-1:0] wr_ptr, rd_ptr;
  logic [max_fifo_addr:0] count;

  always @(posedge intf.clk or negedge intf.rst_n) begin
    if (!intf.rst_n) begin
      wr_ptr <= 0;
    end else if (intf.wr_en && count < FIFO_DEPTH) begin
      mem[wr_ptr] <= intf.data_in;
      intf.wr_ack <= 1;
      wr_ptr <= wr_ptr + 1;
    end else begin
      intf.wr_ack <= 0;
      if (intf.full & intf.wr_en) intf.overflow <= 1;
      else intf.overflow <= 0;
    end
  end

  always @(posedge intf.clk or negedge intf.rst_n) begin
    if (!intf.rst_n) begin
      rd_ptr <= 0;
    end else if (intf.rd_en && count != 0) begin
      intf.data_out <= mem[rd_ptr];
      rd_ptr <= rd_ptr + 1;
    end
  end

  always @(posedge intf.clk or negedge intf.rst_n) begin
    if (!intf.rst_n) begin
      count <= 0;
    end else begin
      if (({intf.wr_en, intf.rd_en} == 2'b10) && !intf.full) count <= count + 1;
      else if (({intf.wr_en, intf.rd_en} == 2'b01) && !intf.empty) count <= count - 1;
    end
  end

  assign intf.full = (count == FIFO_DEPTH) ? 1 : 0;
  assign intf.empty = (count == 0) ? 1 : 0;
  assign intf.underflow = (intf.empty && intf.rd_en) ? 1 : 0;
  assign intf.almostfull = (count == FIFO_DEPTH - 2) ? 1 : 0;
  assign intf.almostempty = (count == 1) ? 1 : 0;
```

```
//ASSERTIONS
  `ifdef SIM
      //asynchronous reset assertion
      always_comb begin
          if(!intf.rst_n)
              arst_sva: assert final(wr_ptr == 0 && rd_ptr == 0 && count == 0);
      end
      //direct assertions
      always_ff @(posedge intf.clk) begin
          assert (wr_ptr < FIFO_DEPTH);
          assert (rd_ptr < FIFO_DEPTH);
          assert (count <= FIFO_DEPTH);
      end
      //assertions properties
      property wr_ack_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) intf.wr_en && count < FIFO_DEPTH |=> intf.wr_ack;
      endproperty
      property overflow_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) intf.wr_en && count == FIFO_DEPTH |=> intf.overflow;
      endproperty
      property underflow_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) intf.rd_en && count == 0 |=> intf.underflow;
      endproperty
      property empty_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) count == 0 |-> intf.empty;
      endproperty
      property full_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) count == FIFO_DEPTH |-> intf.full;
      endproperty
      property almfull_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) count == FIFO_DEPTH - 1 |-> intf.almostfull;
      endproperty
      property almemp_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) count == 1 |-> intf.almostempty;
      endproperty
      property wr_ptr_wrap_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) wr_ptr == 7 && intf.wr_en && count < FIFO_DEPTH |=> wr_ptr == 0;
      endproperty
      property rd_ptr_wrap_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) rd_ptr == 7 && intf.rd_en && count > 0 |=> rd_ptr == 0;
      endproperty
```

```
      //? ASSERTIONS
      wr_ack_sva:              assert property (wr_ack_sva_check);
      overflow_sva:            assert property (overflow_sva_check);
      underflow_sva:           assert property (underflow_sva_check);
      empty_sva:               assert property (empty_sva_check);
      full_sva:                assert property (full_sva_check);
      almfull_sva:             assert property (almfull_sva_check);
      almemp_sva:              assert property (almemp_sva_check);
      wr_ptr_wrap_sva:         assert property (wr_ptr_wrap_sva_check);
      rd_ptr_wrap_sva:         assert property (rd_ptr_wrap_sva_check);
      //? COVER PROPERTIES
      wr_ack_sva_cp:              cover property (wr_ack_sva_check);
      overflow_sva_cp:            cover property (overflow_sva_check);
      underflow_sva_cp:           cover property (underflow_sva_check);
      empty_sva_cp:               cover property (empty_sva_check);
      full_sva_cp:                cover property (full_sva_check);
      almfull_sva_cp:             cover property (almfull_sva_check);
      almemp_sva_cp:              cover property (almemp_sva_check);
      wr_ptr_wrap_sva_cp:         cover property (wr_ptr_wrap_sva_check);
      rd_ptr_wrap_sva_cp:         cover property (rd_ptr_wrap_sva_check);

  `endif
endmodule
```

3. TOP

```
module TOP ();
  bit clk;
  always #10 clk = ~clk;

  FIFO_if intf (clk);
  FIFO DUT (intf);
  TB tb (intf);
  monitor MON (intf);

  always_comb begin
    if (!intf.rst_n)
      arst_sva: assert final(
                    intf.full == 0 &&
                    intf.almostfull == 0 &&
                    intf.empty == 1 &&
                    intf.almostempty == 0 &&
                    intf.overflow == 0 &&
                    intf.underflow == 0 &&
                    intf.data_out == 0 &&
                    intf.wr_ack == 0
                );
  end
endmodule
```

4. Design interface

```
import shared_pkg::*;
interface FIFO_if(clk);
    input bit clk;
    logic rst_n;
    logic wr_en;
    logic rd_en;
    logic wr_ack;
    logic full;
    logic almostfull;
    logic empty;
    logic almostempty;
    logic overflow;
    logic underflow;
    logic [FIFO_WIDTH-1:0] data_in;
    logic [FIFO_WIDTH-1:0] data_out;

    modport DUT (
        input clk,
              rst_n,
              data_in,
              wr_en,
              rd_en,
        output wr_ack,
               full,
               almostfull,
               empty,
               almostempty,
               overflow,
               underflow,
               data_out
    );

    modport TB (
        input clk,
        output rst_n,
               wr_en,
               rd_en,
               data_in
    );

    modport MONITOR (
        input clk,
              rst_n,
              wr_en,
              rd_en,
              wr_ack,
              full,
              almostfull,
              empty,
              almostempty,
              overflow,
              underflow,
              data_in,
              data_out
    );
endinterface
```

5. Shared package

```
package shared_pkg;
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    bit test_finished;
    int error_count = 0;
    int correct_count = 0;
    event TRIG, ACK;
endpackage
```

6. Test bench

```
import FIFO_transaction_pkg::*;
import shared_pkg::*;
module TB(FIFO_if.TB intf);
    FIFO_transaction F_txn;
    initial begin
        F_txn = new();
        intf.rst_n = 0;
        intf.wr_en = 0;
        intf.rd_en = 0;
        intf.data_in = 0;
        @(negedge intf.clk);
        -> TRIG;
        @ACK;
        for(int i=0; i<10000; i++) begin
            assert(F_txn.randomize());
            intf.rst_n = F_txn.rst_n;
            intf.wr_en = F_txn.wr_en;
            intf.rd_en = F_txn.rd_en;
            intf.data_in = F_txn.data_in;
            @(negedge intf.clk);
            -> TRIG;
            @ACK;
        end
        test_finished = 1;
        ->TRIG;
    end
endmodule
```

7. FIFO transaction class

```
package FIFO_transaction_pkg;
    import shared_pkg::*;
    class FIFO_transaction;
        rand logic rst_n;
        rand logic wr_en;
        rand logic rd_en;
        rand logic [FIFO_WIDTH-1:0] data_in;
        logic wr_ack;
        logic full;
        logic almostfull;
        logic empty;
        logic almostempty;
        logic overflow;
        logic underflow;
        logic [FIFO_WIDTH-1:0] data_out;
        int RD_EN_ON_DIST, WR_EN_ON_DIST;

        function new(int RD_EN_ON_DIST=30, int WR_EN_ON_DIST=70);
            this.RD_EN_ON_DIST = RD_EN_ON_DIST;
            this.WR_EN_ON_DIST = WR_EN_ON_DIST;
        endfunction

        constraint c_dist {
            rst_n dist {1:= 98, 0:= 2};
            wr_en dist {0:= 100-WR_EN_ON_DIST, 1:= WR_EN_ON_DIST};
            rd_en dist {0:= 100-RD_EN_ON_DIST, 1:= RD_EN_ON_DIST};
        }

    endclass
endpackage
```

8. FIFO scoreboard

```
package FIFO_scoreboard_pkg;
    import FIFO_transaction_pkg::*;
    import shared_pkg::*;
    class FIFO_scoreboard;
        logic [FIFO_WIDTH-1:0] data_out_ref;
        logic [FIFO_WIDTH-1:0] FIFO_ref [$];
        function void check_data (FIFO_transaction F_txn);
            ref_model(F_txn);
            `ifdef FULL_DEBUG
                if (F_txn.data_out === data_out_ref) begin
                    correct_count++;
                    $display("correct output at time:%0t", $time);
                    $display("input stimulus: write_enable = %b, read_enable = %b, data_in = %0h output = %0h", F_txn.wr_en, F_txn.rd_en, F_txn.data_in, F_txn.data_out);
                    $display("expected current FIFO content:");
                    foreach (FIFO_ref[i]) begin
                        $display("FIFO[%0d] = %0h", i, FIFO_ref[i]);
                    end
                    $display("*****************************************************\n*****************************************************\n");
                end
            `else
                if (F_txn.data_out === data_out_ref) correct_count++;
            `endif
            else begin
                error_count++;
                $display("error at time:%0t, the expected output = %0h, the actual output = %0h",
                $time, data_out_ref, F_txn.data_out);
                $display("input stimulus: write_enable = %b, read_enable = %b, data_in = %0h", F_txn.wr_en, F_txn.rd_en, F_txn.data_in);
                $display("expected current FIFO content:");
                foreach (FIFO_ref[i]) begin
                    $display("FIFO[%0d] = %0h", i, FIFO_ref[i]);
                end
                $display("*****************************************************\n*****************************************************\n");
            end
        endfunction

        function void ref_model (FIFO_transaction F_txn);
            if (!F_txn.rst_n) begin
                FIFO_ref.delete();
                data_out_ref = 0;
            end

            else if(F_txn.wr_en && F_txn.rd_en && FIFO_ref.size() == FIFO_DEPTH)
                data_out_ref = FIFO_ref.pop_front();

            else if(F_txn.wr_en && F_txn.rd_en && FIFO_ref.size() == 0)
                FIFO_ref.push_back(F_txn.data_in);

            else begin
                if(F_txn.wr_en && FIFO_ref.size() < FIFO_DEPTH) FIFO_ref.push_back(F_txn.data_in);
                if(F_txn.rd_en && FIFO_ref.size() > 0) data_out_ref = FIFO_ref.pop_front();
            end
        endfunction
    endclass
```

9. FIFO coverage class

```
package FIFO_coverage_pkg;
    import FIFO_transaction_pkg::*;
    class FIFO_coverage;
        FIFO_transaction F_cvg_txn;

        covergroup CVG;
            wr_en_cp:       coverpoint F_cvg_txn.wr_en;
            rd_en_cp:       coverpoint F_cvg_txn.rd_en;
            wr_ack_cp:      coverpoint F_cvg_txn.wr_ack;
            full_cp:        coverpoint F_cvg_txn.full;
            almost_full_cp: coverpoint F_cvg_txn.almostfull;
            empty_cp:       coverpoint F_cvg_txn.empty;
            almostempty_cp: coverpoint F_cvg_txn.almostempty;
            overflow_cp:    coverpoint F_cvg_txn.overflow;
            underflow_cp:   coverpoint F_cvg_txn.underflow;

            EN_CROSS_WR_ACK:       cross wr_en_cp, rd_en_cp, wr_ack_cp {ignore_bins EN_WR_ACK_IGN = binsof(wr_en_cp) intersect {0} && binsof(wr_ack_cp) intersect {1};}
            EN_CROSS_FULL:         cross wr_en_cp, rd_en_cp, full_cp { ignore_bins EN_FULL_IGN = binsof(rd_en_cp) intersect {1} && binsof(full_cp) intersect {1}; }
            EN_CROSS_ALMOST_FULL:  cross wr_en_cp, rd_en_cp, almost_full_cp;
            EN_CROSS_EMPTY:        cross wr_en_cp, rd_en_cp, empty_cp;
            EN_CROSS_ALMOST_EMPTY: cross wr_en_cp, rd_en_cp, almostempty_cp;
            EN_CROSS_OVERFLOW:     cross wr_en_cp, rd_en_cp, overflow_cp {ignore_bins EN_OVFL_IGN = binsof(wr_en_cp) intersect {0} && binsof(overflow_cp) intersect {1};}
            EN_CROSS_UNDERFLOW:    cross wr_en_cp, rd_en_cp, underflow_cp { ignore_bins EN_UNFL_IGN = binsof(rd_en_cp) intersect {0} && binsof(underflow_cp) intersect {1}; }
        endgroup

        function new ();
            CVG = new();
        endfunction

        function void sample_data(FIFO_transaction F_txn);
            F_cvg_txn = F_txn;
            CVG.sample();
        endfunction
    endclass
endpackage
```

10. Monitor

```
import FIFO_transaction_pkg::*;
import FIFO_scoreboard_pkg::*;
import FIFO_coverage_pkg::*;
import shared_pkg::*;
module monitor(FIFO_if.MONITOR intf);
    FIFO_transaction F_txn;
    FIFO_scoreboard F_scb;
    FIFO_coverage F_cvg;
    initial begin
        F_txn = new();
        F_scb = new();
        F_cvg = new();
        forever begin
            @TRIG;
            F_txn.rst_n       = intf.rst_n;
            F_txn.wr_en       = intf.wr_en;
            F_txn.rd_en       = intf.rd_en;
            F_txn.wr_ack      = intf.wr_ack;
            F_txn.full        = intf.full;
            F_txn.almostfull  = intf.almostfull;
            F_txn.empty       = intf.empty;
            F_txn.almostempty = intf.almostempty;
            F_txn.overflow    = intf.overflow;
            F_txn.underflow   = intf.underflow;
            F_txn.data_in     = intf.data_in;
            F_txn.data_out    = intf.data_out;
            -> ACK;
            fork
                F_cvg.sample_data(F_txn);
                F_scb.check_data(F_txn);
            join
            if(test_finished)begin
                $display("error_count = %d, correct_count = %d", error_count, correct_count);
                $stop;
            end
        end
    end
endmodule
```

11. Do file

```
vlib work
vlog -f src_files.list +cover -covercells +define+SIM
vsim -voptargs=+acc work.TOP -cover
coverage save FIFO.ucdb -onexit
log -r /*
run -all
#quit -sim
#vcover report FIFO.ucdb -du=FIFO -details -annotate -all -output code_cover_rpt.txt
#vcover report FIFO.ucdb -assert -directive -du=FIFO -details -output sva_cover_rpt.txt
#vcover report FIFO.ucdb -package=FIFO_coverage_pkg -cvg -details -annotate -all -output fun_cover_rpt.txt
```

12. Simulation results

- **Assertion results**

Name	Assertion Type	Language	Enable	Failure Count	Pass Count
/TOP/arst_sva	Immediate	SVA	on	182	0
/TOP/DUT/arst_sva	Immediate	SVA	on	0	1
/TOP/DUT/wr_ack_sva	Concurrent	SVA	on	0	1
/TOP/DUT/overflow_sva	Concurrent	SVA	on	0	1
/TOP/DUT/underflow_sva	Concurrent	SVA	on	90	1
/TOP/DUT/empty_sva	Concurrent	SVA	on	0	1
/TOP/DUT/full_sva	Concurrent	SVA	on	0	1
/TOP/DUT/almfull_sva	Concurrent	SVA	on	1198	0
/TOP/DUT/almemp_sva	Concurrent	SVA	on	0	1
/TOP/DUT/wr_ptr_wrap_sva	Concurrent	SVA	on	0	1
/TOP/DUT/rd_ptr_wrap_sva	Concurrent	SVA	on	0	1
/TOP/DUT/#ublk#306607#58/immed__59	Immediate	SVA	on	0	1
/TOP/DUT/#ublk#306607#58/immed__60	Immediate	SVA	on	0	1
/TOP/DUT/#ublk#306607#58/immed__61	Immediate	SVA	on	0	1
/TOP/tb/#anonblk#1410#14#4#/#ublk#1410#14/immed__15	Immediate	SVA	on	0	1

- **Output against golden model summary results**

```
# ************************************************************
#
# error_count =        5344, correct_count =        4658
# ** Note: $stop    : monitor.sv(34)
#    Time: 200020 ns  Iteration: 4  Instance: /TOP/MON
# Break in Module monitor at monitor.sv line 34
```

- **An error stimulus test case**

```
# correct output at time:199600
# input stimulus: write_enable = 1, read_enable = 0, data_in = 61ac output = b746
# expected current FIFO content:
# FIFO[0] = b639
# FIFO[1] = 1075
# FIFO[2] = 5882
# FIFO[3] = cdfd
# FIFO[4] = 3ccb
# FIFO[5] = 61ac
# ********************************************************
# ********************************************************
#
# correct output at time:199620
# input stimulus: write_enable = 1, read_enable = 0, data_in = 2571 output = b746
# expected current FIFO content:
# FIFO[0] = b639
# FIFO[1] = 1075
# FIFO[2] = 5882
# FIFO[3] = cdfd
# FIFO[4] = 3ccb
# FIFO[5] = 61ac
# FIFO[6] = 2571
# ********************************************************
# ********************************************************
#
# correct output at time:199640
# input stimulus: write_enable = 0, read_enable = 0, data_in = c908 output = b746
# expected current FIFO content:
# FIFO[0] = b639
# FIFO[1] = 1075
# FIFO[2] = 5882
# FIFO[3] = cdfd
# FIFO[4] = 3ccb
# FIFO[5] = 61ac
# FIFO[6] = 2571
# ********************************************************
# ********************************************************
#
# correct output at time:199660
# input stimulus: write_enable = 1, read_enable = 0, data_in = a56d output = b746
# expected current FIFO content:
# FIFO[0] = b639
# FIFO[1] = 1075
# FIFO[2] = 5882
# FIFO[3] = cdfd
# FIFO[4] = 3ccb
# FIFO[5] = 61ac
# FIFO[6] = 2571
# FIFO[7] = a56d
# ********************************************************
# ********************************************************
```

```
# correct output at time:199680
# input stimulus: write_enable = 1, read_enable = 0, data_in = 4deb output = b746
# expected current FIFO content:
# FIFO[0] = b639
# FIFO[1] = 1075
# FIFO[2] = 5882
# FIFO[3] = cdfd
# FIFO[4] = 3ccb
# FIFO[5] = 61ac
# FIFO[6] = 2571
# FIFO[7] = a56d
# ********************************************************
# ********************************************************
#
# error at time:199700, the expected output = b639, the actual output = 4deb
# input stimulus: write_enable = 1, read_enable = 1, data_in = ba87
# expected current FIFO content:
# FIFO[0] = 1075
# FIFO[1] = 5882
# FIFO[2] = cdfd
# FIFO[3] = 3ccb
# FIFO[4] = 61ac
# FIFO[5] = 2571
# FIFO[6] = a56d
# ********************************************************
# ********************************************************
```

13. Detected bugs

- **Output data**

  - **observation**

    When the FIFO is full and the input write enable and read enable signals are both high the output becomes different from the expected output.

  - **Cause**

    This scenario is not considered when updating the FIFO's internal counter. When both write and read enables are high, the counter value remains unchanged, rather than decrementing by one—even though only a read operation should be effectively counted in that case. Similarly, when the FIFO is empty and both read and write enables are high, the counter value remains zero instead of incrementing by one, even though only a write operation should be considered in this situation.

```
always @(posedge intf.clk or negedge intf.rst_n) begin
  if (!intf.rst_n) begin
    count <= 0;
  end else begin
    if (({intf.wr_en, intf.rd_en} == 2'b10) && !intf.full) count <= count + 1;
    else if (({intf.wr_en, intf.rd_en} == 2'b01) && !intf.empty) count <= count - 1;
  end
end
```

  - **FIX**

```
always @(posedge intf.clk or negedge intf.rst_n) begin
  if (!intf.rst_n) begin
    count <= 0;
  end else begin
    if (({intf.wr_en, intf.rd_en} == 2'b10) && !intf.full) count <= count + 1;
    else if (({intf.wr_en, intf.rd_en} == 2'b01) && !intf.empty) count <= count - 1;
    else if (({intf.wr_en, intf.rd_en} == 2'b11) && intf.empty) count <= count + 1;
    else if (({intf.wr_en, intf.rd_en} == 2'b11) && intf.full) count <= count - 1;
  end
end
```

- **Under flow signal**

  - **Observation**

    Under flow signal is driven computationally not sequentially

  - **Error code**

```
assign intf.full = (count == FIFO_DEPTH) ? 1 : 0;
assign intf.empty = (count == 0) ? 1 : 0;
assign intf.underflow = (intf.empty && intf.rd_en) ? 1 : 0;
assign intf.almostfull = (count == FIFO_DEPTH - 2) ? 1 : 0;
assign intf.almostempty = (count == 1) ? 1 : 0;
```

❖ **FIX**

```
always @(posedge intf.clk or negedge intf.rst_n) begin
  if (!intf.rst_n) begin
    rd_ptr <= 0;
    intf.data_out <= 0;
    intf.underflow <= 0;
  end else if (intf.rd_en && count != 0) begin
    intf.data_out <= mem[rd_ptr];
    rd_ptr <= rd_ptr + 1;
  end else begin
      if(intf.rd_en && intf.empty) intf.underflow <= 1;
      else intf.underflow <= 0;
  end
end
```

- **Almost full signal**

❖ **Observation**

It was found that the *almost full* signal is asserted one entry earlier than expected. When the FIFO still has two available slots, the signal is raised instead of waiting until only one slot remains before becoming full.

❖ **Error in the code**

```
assign intf.full = (count == FIFO_DEPTH) ? 1 : 0;
assign intf.empty = (count == 0) ? 1 : 0;
assign intf.underflow = (intf.empty && intf.rd_en) ? 1 : 0;
assign intf.almostfull = (count == FIFO_DEPTH - 2) ? 1 : 0;
assign intf.almostempty = (count == 1) ? 1 : 0;
```

❖ **FIX**

```
assign intf.full = (count == FIFO_DEPTH) ? 1 : 0;
assign intf.empty = (count == 0) ? 1 : 0;
assign intf.almostfull = (count == FIFO_DEPTH - 1) ? 1 : 0;
assign intf.almostempty = (count == 1) ? 1 : 0;
```

- **Top resetting**

❖ **Observation**

The registered outputs are not reset when the reset signal is asserted, it remains unchanged.

❖ **Fix**

Reset output registered signals which are data_out, wr_ack, overflow, underflow.

14. Correct design

```
import shared_pkg::*;
module FIFO (
    FIFO_if.DUT intf
);
  localparam max_fifo_addr = $clog2(FIFO_DEPTH);

  logic [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];

  logic [max_fifo_addr-1:0] wr_ptr, rd_ptr;
  logic [max_fifo_addr:0] count;

  always @(posedge intf.clk or negedge intf.rst_n) begin
    if (!intf.rst_n) begin
      wr_ptr <= 0;
      intf.overflow <= 0;
      intf.wr_ack <= 0;
    end else if (intf.wr_en && count < FIFO_DEPTH) begin
      mem[wr_ptr] <= intf.data_in;
      intf.wr_ack <= 1;
      wr_ptr <= wr_ptr + 1;
    end else begin
      intf.wr_ack <= 0;
      if (intf.full & intf.wr_en) intf.overflow <= 1;
      else intf.overflow <= 0;
    end
  end

  always @(posedge intf.clk or negedge intf.rst_n) begin
    if (!intf.rst_n) begin
      rd_ptr <= 0;
      intf.data_out <= 0;
      intf.underflow <= 0;
    end else if (intf.rd_en && count != 0) begin
      intf.data_out <= mem[rd_ptr];
      rd_ptr <= rd_ptr + 1;
    end else begin
        if(intf.rd_en && intf.empty) intf.underflow <= 1;
        else intf.underflow <= 0;
    end
  end

  always @(posedge intf.clk or negedge intf.rst_n) begin
    if (!intf.rst_n) begin
      count <= 0;
    end else begin
      if (({intf.wr_en, intf.rd_en} == 2'b10) && !intf.full) count <= count + 1;
      else if (({intf.wr_en, intf.rd_en} == 2'b01) && !intf.empty) count <= count - 1;
      else if (({intf.wr_en, intf.rd_en} == 2'b11) && intf.empty) count <= count + 1;
      else if (({intf.wr_en, intf.rd_en} == 2'b11) && intf.full) count <= count - 1;
    end
  end
```

```
assign intf.full = (count == FIFO_DEPTH) ? 1 : 0;
assign intf.empty = (count == 0) ? 1 : 0;
assign intf.almostfull = (count == FIFO_DEPTH - 1) ? 1 : 0;
assign intf.almostempty = (count == 1) ? 1 : 0;

//ASSERTIONS
  `ifdef SIM
      //asynchronous reset assertion
      always_comb begin
          if(!intf.rst_n)
              arst_sva: assert final(wr_ptr == 0 && rd_ptr == 0 && count == 0);
      end
      //direct assertions
      always_ff @(posedge intf.clk) begin
          assert (wr_ptr < FIFO_DEPTH);
          assert (rd_ptr < FIFO_DEPTH);
          assert (count <= FIFO_DEPTH);
      end
      //assertions properties
      property wr_ack_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) intf.wr_en && count < FIFO_DEPTH |=> intf.wr_ack;
      endproperty
      property overflow_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) intf.wr_en && count == FIFO_DEPTH |=> intf.overflow;
      endproperty
      property underflow_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) intf.rd_en && count == 0 |=> intf.underflow;
      endproperty
      property empty_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) count == 0 |-> intf.empty;
      endproperty
      property full_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) count == FIFO_DEPTH |-> intf.full;
      endproperty
      property almfull_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) count == FIFO_DEPTH - 1 |-> intf.almostfull;
      endproperty
      property almemp_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) count == 1 |-> intf.almostempty;
      endproperty
      property wr_ptr_wrap_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) wr_ptr == 7 && intf.wr_en && count < FIFO_DEPTH |=> wr_ptr == 0;
      endproperty
      property rd_ptr_wrap_sva_check;
          @(posedge intf.clk) disable iff(!intf.rst_n) rd_ptr == 7 && intf.rd_en && count > 0 |=> rd_ptr == 0;
      endproperty

      //? ASSERTIONS
      wr_ack_sva:              assert property (wr_ack_sva_check);
      overflow_sva:            assert property (overflow_sva_check);
      underflow_sva:           assert property (underflow_sva_check);
      empty_sva:               assert property (empty_sva_check);
      full_sva:                assert property (full_sva_check);
      almfull_sva:             assert property (almfull_sva_check);
      almemp_sva:              assert property (almemp_sva_check);
      wr_ptr_wrap_sva:         assert property (wr_ptr_wrap_sva_check);
      rd_ptr_wrap_sva:         assert property (rd_ptr_wrap_sva_check);
      //? COVER PROPERTIES
      wr_ack_sva_cp:              cover property (wr_ack_sva_check);
      overflow_sva_cp:            cover property (overflow_sva_check);
      underflow_sva_cp:           cover property (underflow_sva_check);
      empty_sva_cp:               cover property (empty_sva_check);
      full_sva_cp:                cover property (full_sva_check);
      almfull_sva_cp:             cover property (almfull_sva_check);
      almemp_sva_cp:              cover property (almemp_sva_check);
      wr_ptr_wrap_sva_cp:         cover property (wr_ptr_wrap_sva_check);
      rd_ptr_wrap_sva_cp:         cover property (rd_ptr_wrap_sva_check);

  `endif
endmodule
```

15. Simulation results after modification

```
# ************************************************************
#
# error_count =          0, correct_count =       10002
# ** Note: $stop    : monitor.sv(34)
#    Time: 200020 ns  Iteration: 4  Instance: /TOP/MON
# Break in Module monitor at monitor.sv line 34

VSIM 2>
```

Name	Assertion Type	Language	Enable	Failure Count	Pass Count
/TOP/arst_sva	Immediate	SVA	on	0	1
/TOP/DUT/arst_sva	Immediate	SVA	on	0	1
/TOP/DUT/wr_ack_sva	Concurrent	SVA	on	0	1
/TOP/DUT/overflow_sva	Concurrent	SVA	on	0	1
/TOP/DUT/underflow_sva	Concurrent	SVA	on	0	1
/TOP/DUT/empty_sva	Concurrent	SVA	on	0	1
/TOP/DUT/full_sva	Concurrent	SVA	on	0	1
/TOP/DUT/almfull_sva	Concurrent	SVA	on	0	1
/TOP/DUT/almemp_sva	Concurrent	SVA	on	0	1
/TOP/DUT/wr_ptr_wrap_sva	Concurrent	SVA	on	0	1
/TOP/DUT/rd_ptr_wrap_sva	Concurrent	SVA	on	0	1
/TOP/DUT/#ublk#306607#66/immed__67	Immediate	SVA	on	0	1
/TOP/DUT/#ublk#306607#66/immed__68	Immediate	SVA	on	0	1
/TOP/DUT/#ublk#306607#66/immed__69	Immediate	SVA	on	0	1
/TOP/tb/#anonblk#1410#14#4#/#ublk#1410#14/immed__15	Immediate	SVA	on	0	1

16. Coverage results

- **Code coverage**

```
Coverage Report Summary Data by file

================================================================================
=== File: FIFO_correct.sv
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        27        27         0   100.00%
    Statements                      30        30         0   100.00%
    Toggles                         20        20         0   100.00%

================================================================================
```

- **Assertion coverage**

Cover Directives

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included
/TOP/DUT/wr_ack_sva_cp	SVA	✓	Off	3868	1	Unlimited	1	100%		✓
/TOP/DUT/overflow_sva_cp	SVA	✓	Off	2803	1	Unlimited	1	100%		✓
/TOP/DUT/underflow_sva_cp	SVA	✓	Off	96	1	Unlimited	1	100%		✓
/TOP/DUT/empty_sva_cp	SVA	✓	Off	335	1	Unlimited	1	100%		✓
/TOP/DUT/full_sva_cp	SVA	✓	Off	4132	1	Unlimited	1	100%		✓
/TOP/DUT/almfull_sva_cp	SVA	✓	Off	2552	1	Unlimited	1	100%		✓
/TOP/DUT/almemp_sva_cp	SVA	✓	Off	465	1	Unlimited	1	100%		✓
/TOP/DUT/wr_ptr_wrap_sva_cp	SVA	✓	Off	405	1	Unlimited	1	100%		✓
/TOP/DUT/rd_ptr_wrap_sva_cp	SVA	✓	Off	267	1	Unlimited	1	100%		✓

- **Functional coverage**

Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/FIFO_coverage_pkg/FIFO_coverage		100.00%				
TYPE CVG		100.00%	100	100.00...		✓
CVP CVG::wr_en_cp		100.00%	100	100.00...		✓
CVP CVG::rd_en_cp		100.00%	100	100.00...		✓
CVP CVG::wr_ack_cp		100.00%	100	100.00...		✓
CVP CVG::full_cp		100.00%	100	100.00...		✓
CVP CVG::almost_full_cp		100.00%	100	100.00...		✓
CVP CVG::empty_cp		100.00%	100	100.00...		✓
CVP CVG::almostempty_cp		100.00%	100	100.00...		✓
CVP CVG::overflow_cp		100.00%	100	100.00...		✓
CVP CVG::underflow_cp		100.00%	100	100.00...		✓
CROSS CVG::EN_CROSS_WR_ACK		100.00%	100	100.00...		✓
CROSS CVG::EN_CROSS_FULL		100.00%	100	100.00...		✓
CROSS CVG::EN_CROSS_ALMOST_FULL		100.00%	100	100.00...		✓
CROSS CVG::EN_CROSS_EMPTY		100.00%	100	100.00...		✓
CROSS CVG::EN_CROSS_ALMOST_EMPTY		100.00%	100	100.00...		✓
CROSS CVG::EN_CROSS_OVERFLOW		100.00%	100	100.00...		✓
CROSS CVG::EN_CROSS_UNDERFLOW		100.00%	100	100.00...		✓